

# Development Log — Syllabus → Textbook Page Mapper

This documents every iteration of this pipeline: what was built, what broke, how each bug was root-caused, and how it was verified fixed. Kept alongside the code so future changes (or a different college's syllabus format) can be debugged the same way.

Test file used throughout: `AI_DS.pdf` (Sri Sairam Engineering College, AI&DS curriculum regulation document, 54 subjects).

---

## Round 1 — Initial build

**Goal:** parse a syllabus PDF into Subject → Module → Topic, map each topic to textbook page ranges, output CSV.

### Built:

- `syllabus_parser.py` — regex parser: subject header (CODE NAME L T P C), UNIT <roman> <title> <hours>, TEXT BOOKS/REFERENCE BOOKS blocks.
- `book_indexer.py` — per-page text extraction (PyMuPDF/pdfplumber), with OCR fallback (pdf2image + pytesseract) for pages with < 30 chars of extracted text (i.e. scanned pages).
- `topic_mapper.py` — two-stage topic→page mapping: (1) fuzzy-match candidate page retrieval, no LLM, (2) LLM adjudication over only those candidate excerpts to pick `page_start/page_end/chapter/coverage_notes`. Backend was Ollama initially.
- `main.py` — orchestrator, writes final CSV.

**Verification:** parsed `AI_DS.pdf` → 54 subjects found. Spot-checked one subject's units/topics printed correctly. Generated a synthetic textbook PDF with `reportlab` and ran the full pipeline end-to-end (no real book was uploaded yet) to confirm the CSV write path worked.

**Known gap at this point:** topics were a flat list (no sub-topic level), and `split_topics()` used naive comma/semicolon splitting.

---

## Round 2 — Sub-topics, Gemini backend

**User ask:** separate topics \*and\* sub-topics, each with its own page range; recommend Gemini vs Ollama.

**Changes:**

- `TopicItem(topic, subtopics: List[str])` replaced the flat topic list. `split_topics()` began splitting on ; for topic boundaries and , within a segment for sub-topics.
- Added `call_llm_gemini()` in `topic_mapper.py`, made `gemini-2.5-flash` the default backend (better accuracy on noisy/OCR'd excerpts than a local 8B Ollama model, cheap at this call volume, no local GPU needed). Ollama and an OpenAI-compatible backend kept as alternatives.
- `main.py` updated to iterate topic → sub-topic and emit one CSV row per sub-topic (falling back to one row per topic when a topic had no sub-topics).

**Verification:** unit-tested `split_topics()` against a synthetic Python-syllabus paragraph; confirmed topic label wasn't duplicated as its own first sub-topic (an off-by-one in the "long segment" branch — fixed by excluding `subs[0]` from the sub-topics list when it was reused as the topic label). Ran the full CSV-writing loop end-to-end with a mocked `parse_syllabus()` and a small synthetic book, confirmed `id` values like `GE_M2_T1_S1` and per-sub-topic page ranges appeared correctly.

---

## Round 3 — OmniRoute integration

**User ask:** add OmniRoute (multi-provider AI gateway) as a backend.

**Change:** since OmniRoute exposes an OpenAI-compatible endpoint, added `call_llm_omniroute()` as a thin wrapper around the existing `call_llm_openai_compatible()`, pointed at `OMNIROUTE_BASE_URL` (default `http://localhost:20128/v1`) with `OMNIROUTE_API_KEY`. No new parsing/adjudication logic needed — reused the already-tested OpenAI-compatible code path.

**Verification:** compiled clean; couldn't run a live call in the sandbox (no `openai` package installed there, no local OmniRoute instance to hit), so this was verified by code review — it's the same client code already exercised for `--backend openai`, just with a different `base_url`.

---

## Round 4 — Real syllabus + reference image uploaded

**User uploaded:** `AI_DS.pdf` and a screenshot of the target CSV column layout (``id, subject_code, subject_name, module_no, module_title, topic, sub_topic, book_title, author, edition, publisher, year, book_type, chapter, page_start, page_end, coverage_notes``).

### Findings:

- Column layout already matched what `main.py` was producing — no schema change needed.
- Ran the real parser against `AI_DS.pdf` and found the subject shown in the reference image: IT4351 — Data Structures Using Python, with the same three textbook citations shown in the screenshot (Goodrich et al. 2013, Lee & Hubbard 2015, Necaie 2011).
  - **New bug found:** citation author names had lost their spaces — "MichaelT.Goodrich,RobertoTamassia,andMichaelH.Goldwasser". Root cause: the \*source PDF itself\* has this defect in that specific citation line (confirmed via raw pdfplumber extraction — not an extraction artifact, the text layer itself is missing spaces there; a second citation two lines below extracted with normal spacing).
  - **Fix:** added `_fix_missing_spaces()` — inserts a space at lowercase→Uppercase and letter→digit boundaries. Recovers author-name boundaries well; cannot recover spaces lost between two lowercase words (e.g. "Algorithmsin" → "Algorithms in" would need a dictionary) — documented as a known limitation rather than silently guessing wrong.
  - Pre-filled `book_files.json` for IT4351 with the correct citation metadata (author/title/publisher/year matching the reference image), leaving file paths as placeholders pending the actual book upload.

**Verification:** re-ran citation parsing for IT4351 before/after the fix and confirmed author strings were readable.

---

## Round 5 — Systematic bug report from real output review

The user reviewed real parser output against the source PDF and reported six distinct problems. Each was root-caused against the actual `AI_DS.pdf` text (not assumed) before fixing.

## 5.1 — "Modules truncate to ~5 topics"

**Root cause:** not a real data-loss bug. The CLI demo block at the bottom of `syllabus_parser.py` printed `for t in m.topics[:5]:` — a display slice left over from an early debug print, not a limit on what was parsed or stored. `len(m.topics)` for the modules named in the report (PH4103 Module 1, etc.) was already 10+ in the underlying data.

**Fix:** removed all `[:5]/[:70]` slicing from the CLI demo output; added an optional subject-code argument (``python syllabus_parser.py file.pdf HS4101``) to print one subject's \*entire\* structure for exactly this kind of verification, instead of only ever showing the first 2 subjects.

## 5.2 — Continuation lines split into duplicate/fragmented topics

**Root cause:** real bug. `split_topics()` split on `;` for topics, then `,` for sub-topics, with no awareness that a colon (``Torsion pendulum: theory and experiment``) introduces a sub-list rather than a new top-level topic. "conditionals – if, unless, in case, when and others" was being comma-split into four sibling fragments instead of staying grouped as one topic's sub-topics.

**Fix:** rewrote the technical-unit splitter (`_split_topics_general()`) to treat a colon inside a segment as `label: sub-topic, sub-topic, ...` rather than a topic boundary.

## 5.3 — "PDF line breaks read as topic boundaries mid-sentence"

**Root cause:** investigated directly — line breaks were already being collapsed to spaces (`re.sub(r'\s+', ' ', text)`) before any splitting occurred, so a literal PDF line break was never itself a split point. The symptom the user saw was actually 5.2 (over-eager comma splitting) wearing a different hat — e.g. ``"population of energy levels, Einstein's A and B coefficients derivation"`` looked line-break-triggered but was actually comma-triggered. Confirmed by testing the raw (line-break-containing) UNIT II text directly through `split_topics()` before touching any line-break-specific code, and observing the same fragmentation pattern predicted by 5.2's cause.

**Fix:** covered by the 5.2 fix; no separate line-break-handling logic was needed once colon/comma handling was corrected.

## 5.4 — Modules losing trailing topics (HS4101 Module 5, PH4103 Modules 4–5)

**Root cause:** two contributing factors, both confirmed against raw text:

(a) the same CLI-slicing issue as 5.1 (module 5 exists further down the printed list, so [ : 2 ] subject-level slicing plus [ : 5 ] topic-level slicing compounded to make it *\*look\** dropped);

(b) a **real** trailing-text bug: TOTAL\s\*:\s\* regex required no space before the colon, but the source PDF has "TOTAL : 45 PERIODS" (space before colon). This let "...Applications of Single crystal (Qualitative). TOTAL : 45 PERIODS" leak into PH4103 Module 5's last topic as garbage trailing text.

**Fix:** TOTAL\s\*:\s\*\d+\s\*PERIODS.\*\$ (optional whitespace before the colon) plus the CLI-slicing fix from 5.1.

## 5.5 — Wrong semester (HS4101, PH4103 reported as "Sem VIII" instead of "Sem I")

**Root cause:** confirmed via direct regex testing against the extracted full text. The source PDF renders "SEMESTER I" through "SEMESTER VII" with a **space**, but "SEMESTER-I" through "SEMESTER-VII" (en dash, no space) in a different table earlier in the document, and "SEMESTER VIII" appears in **both** styles at different points. The original semester\_for\_pos() logic picked "the nearest SEMESTER heading before this position, scanning the entire document" — since the en-dash style didn't match the original regex (SEMESTER\s+), Semester I's heading was invisible to the matcher, so by the time the matcher reached HS4101/PH4103 (physically located after all 8 semesters' overview headings in the document), the last regex-visible heading it had passed was a `SEMESTER VIII` occurrence from an earlier overview table — wrongly stamping Sem I subjects as Sem VIII.

**Fix:** replaced positional nearest-heading lookup with build\_semester\_map() — for each SEMESTER <roman> heading (now matching both space and en-dash styles), scan **only that heading's own table span** for [A-Z]{2,4}\d{3,5} course codes and record code → semester, keeping the *\*first\** assignment per code (guards against a later cumulative/summary table, if any, overwriting a correct earlier one). Added a size guard (MAX\_SEMESTER\_TABLE\_CHARS) to skip any span longer than a real per-semester table would be, in case a later heading's "span" accidentally swept up unrelated content.

**Verification:** HS4101 → Sem I, PH4103 → Sem I, IT4351 → Sem III (expected — Data Structures is a 3rd-semester course), IT4454 → Sem IV. Ran a full 54-subject scan afterward; 22 codes came back with no semester, all of which are open/professional electives that live in a flexible elective basket rather than a fixed per-semester table in the source PDF —

confirmed this is correct-and-honest behavior, not a new bug, and documented it in the README rather than guessing a semester for them.

## 5.6 — Inconsistent topic granularity ("Reading" alone vs. Writing/Listening/etc. merged unpredictably)

**Root cause:** real structural bug. Communicative-English-style units are written as a sequence of skill-labels (Reading, Writing, Listening, Speaking, Language development, Vocabulary development) each introducing a clause, but punctuated with a mix of :, -, —, and . inconsistently — there's no single reliable delimiter. The old punctuation-only splitter had no concept of these labels as boundaries, so "Reading" (appearing with almost nothing after it before the next delimiter) became its own near-empty topic while Writing/Listening/etc. got merged into whatever fragment the punctuation happened to produce.

**Fix:** added a section-aware mode. If a unit's body contains 3+ matches of the known skill-section keywords, `_split_topics_by_section()` is used instead of the generic splitter: each keyword occurrence starts a new Topic, and everything up to the next keyword occurrence becomes that topic's sub-topics (re-using the same dash/comma phrase splitter used elsewhere). Non-language (technical) units, which don't hit 3+ keyword matches, still use the punctuation-based splitter.

**Follow-up bug found during this fix's own verification:** the source text for HS4101 Unit III literally contains "...jumbled sentences. Listening:\nListening to ted talks..." — the word "Listening" appears twice back-to-back in the source itself (a duplicate-label quirk, not a parsing bug). The section splitter matched both occurrences, producing an empty first "Listening" topic that then got folded into the \*previous\* topic's sub-topics via the orphan-merge step, resulting in a stray "Listening" bullet appearing under the Writing topic.

**Fix:** section boundaries with an empty body (i.e. the very next token is another section keyword) are now skipped entirely rather than kept-then-merged, so duplicate/empty labels produce no output at all instead of noise.

**Regression check after all of Round 5's fixes:** re-ran IT4351 (Data Structures — a technical-unit-style subject already validated in Round 4) to confirm the section-aware/colon-aware rewrite hadn't broken the previously-working path. Also ran the full pipeline end-to-end (synthetic book) after the parser rewrite to confirm `main.py/topic_mapper.py` still consumed the new `TopicItem` structure correctly.

---

## Current status

- `syllabus_parser.py`: subject/module/topic/sub-topic/citation/semester

extraction verified against real syllabus text for both unit styles

(language-skill and technical), with root-caused fixes for all 6 reported issues plus regression coverage.

- `book_indexer.py`, `topic_mapper.py`, `main.py`: unchanged in this round;

last verified end-to-end with a synthetic textbook PDF (no real textbook has been uploaded yet).

- **Not yet tested:** the topic-mapper's LLM adjudication step against a \*real\* textbook (all LLM-backend runs so far have exercised the heuristic fallback path only, since no Gemini/Ollama/OmniRoute credential or local model is available in this sandbox) — this needs verification once real book files and an API key/local model are available on the user's machine.